

THE FOREVER TREASURE

Cinematic Animation Rebuild

Implementation-Grade Codex Prompt

GSAP | StPageFlip | Motion | Rive | Lottie

Prepared for Kato Gray - July 17, 2026

A single, copyable specification for the complete animation architecture, visual-content expansion, implementation, testing, and development showcase.

COPYING NOTE

This PDF contains selectable text. A companion Markdown file is also provided so the entire prompt can be copied as one uninterrupted block without headers or page numbers.

Contents Overview

- Task goals, repository safety, and current weaknesses
- Dependency migration and five-library architecture
- AnimationDirector, server synchronization, ownership, and motion modes
- GSAP, StPageFlip, Motion, Rive, and Lottie implementation rules
- Asset system and new visual content
- Detailed page-by-page implementation for all player and GM surfaces
- Audio synchronization
- Development-only TEST ANIMATIONS showcase and cinematic trailer
- Component decomposition, performance, accessibility, and testing
- Required deliverables, quality bar, and final validation receipt

PRIMARY DIRECTIVE

Do not treat this as an installation task. The goal is a physical, cinematic, truthful, accessible, and testable rebuild of the current animation system.

BEGIN CODEX PROMPT

Forever Treasure Companion - Full Cinematic Animation Rebuild

You are working in the canonical GitHub repository:

- Repository: `Kgray44/treasurehuntSoT`
- Canonical branch: `main`
- Application: The Forever Treasure Companion
- Framework: Next.js App Router, React, strict TypeScript

This task is a complete animation architecture and cinematic-polish rebuild. It is not a superficial animation pass. Do not merely install packages, lengthen CSS transitions, rename stages, or add more fades. The current application already has strong narrative ideas, but many animations are timer-driven class changes and whole-block opacity transitions. Replace that weak implementation with coordinated, physical, detailed, testable animation systems.

The final experience must feel like a handcrafted nautical-fantasy story companion: a physical journal, an alive voyage chart, a relic altar, a side-quest ledger, a captain's log, and a dormant celestial finale mechanism. It must remain usable, accessible, secure, responsive, and truthful about authoritative server state.

Do not ask for clarification. Inspect the repository, resolve implementation details from the existing architecture and official library documentation, make sensible decisions, and complete the best production-quality implementation possible.



1. Non-negotiable task goals

Implement and use all of the following technologies for the work each is best suited to perform:

1. GSAP
2. StPageFlip
3. Motion, meaning the modern `motion` package that replaces `framer-motion`
4. Rive
5. Lottie

The five systems must cooperate under a documented ownership architecture. Do not make all five libraries animate the same DOM properties. Each element must have one animation owner for transforms, opacity, and geometry at any given time.

The hierarchy is:

- GSAP is the cinematic director and sequence orchestrator.
- Motion is the application-interface movement engine.
- StPageFlip is the physical book and page-turning engine.
- Rive supplies stateful, interactive hero objects.
- Lottie supplies ambient loops and pre-authored visual effects.

The rebuild must include:

- New animation infrastructure
- New visual content and physical scene elements where the existing page is too sparse to support a convincing animation
- A replacement for timer-only cinematic sequencing
- Proper cross-library synchronization
- Robust lifecycle cleanup
- Full, gentle, and reduced-motion modes
- Development-only animation testing tools
- A development button labeled `TEST ANIMATIONS`

- A complete non-destructive animation showcase and cinematic trailer
- Unit, component, integration, end-to-end, visual, accessibility, performance, and cleanup testing
- Documentation of architecture, assets, ownership, limitations, and validation results

Do not stop after installing dependencies or producing scaffolding. Implement meaningful, visible examples of each library and integrate them into the actual current pages wherever valid assets permit.



2. Repository safety, privacy, and workflow

2.1 Start by inspecting reality

Before changing code:

1. Fetch remote changes.
2. Inspect the current branch, working tree, remote, and latest commit.
3. Verify whether `main` is clean and current.
4. Pull with rebase only if the working tree is clean.
5. Inspect `package.json`, lockfile, current Next.js configuration, current animation components, `src/app/globals.css`, tests, public assets, and documentation.
6. Reconfirm current repository visibility.
7. Search the complete repository for:

```
- `framer-motion`
- `motion-dom`
- `motion-utils`
- `setTimeout` animation staging
- `useCinematicTransition`
- `useCeremony`
- `AnimatePresence`
- `.riv`
- Lottie JSON files
- SVG route or handwriting assets
- current audio code
- current development animation lab
```

Treat paths in this prompt as expected current locations, not unquestionable truth. If the repository changed, adapt while preserving the intent.

2.2 Protect surprise content

The repository has been public during prior work. Until visibility is explicitly verified private:

- Do not add real romantic surprise content.
- Do not add Sera's private information.
- Do not add final clues, final locations, final ending copy, personal photographs, secret passwords, production access codes, or actual finale content.
- Continue to use generic development content.
- Do not copy Sea of Thieves art, UI, map assets, sound, fonts, or copyrighted material.
- Use original nautical-fantasy visual content, user-created assets already licensed for the project, or clearly licensed dependencies.
- Record every external visual or audio asset in the existing asset-license documentation.

Never hide secret content in the DOM, JavaScript bundles, Rive inputs, Lottie JSON, comments, source maps, network responses, or development showcase.

2.3 Preserve application truth and security

Animations must never lie about state.

- Do not reveal a chapter before the server confirms release.
- Do not show an artifact as awarded before the authoritative action succeeds.

- Do not display a map route before the server response is successful.
- Do not claim authentication success before authentication succeeds.
- Do not let skipping an animation bypass security, mutation confirmation, or state gating.
- Preserve CSRF protection, session security, rate limiting, allowlist serialization, SSE ordering, audit logs, undo points, and current transaction semantics.
- Preserve all current public-snapshot privacy boundaries.
- Preserve keyboard accessibility and no-sign-in player behavior where applicable.

2.4 Versioning and Git

Follow the repository's documented workflow.

- Work from an intentional feature branch unless the repository workflow explicitly requires direct `main` work.
- Do not force-push.
- Do not rewrite unrelated history.
- Do not include unrelated changes.
- Make focused commits by major milestone if allowed by the workflow.
- Run the complete validation gate before final commit and push.
- Update version metadata using the project's existing versioning convention. Apply only the increment justified by this work.
- Push the completed branch and verify remote parity if the repository workflow expects pushing.
- Provide exact commit SHA, branch, version, validation results, and changed-file summary in the final receipt.



3. Current implementation weaknesses to replace

The current project uses Next.js, React, Tailwind, Framer Motion, CSS animations, and custom timers. Existing sequence names such as `dark-sea`, `tide-arrives`, `title-written`, `seal-breaking`, `hidden-ink`, `route-drawing`, and `relic-emerging` express the right story ideas, but the implementation frequently performs this pattern:

1. Set a stage string.
2. Wait a fixed duration.
3. Apply a CSS class.
4. Advance to the next stage.

The chapter ceremony similarly uses a timer queue for `omen`, `attention`, `seal`, `parchment`, `ink-heading`, `ink-story`, `ink-objective`, `ink-riddle`, `map`, and `active`.

Replace that with real animation timelines and actual completion-driven sequencing.

The current implementation also contains these weaknesses:

- Whole paragraphs mostly fade in rather than appearing as ink or writing.
- The page release is a small `rotateY` motion rather than a physical page turn.
- Main section changes often mount or disappear without a meaningful transition.
- The Quartermaster action overlay shows essentially one seal and a stage-name string for many unrelated actions.
- The artifact altar is mostly abstract buttons and placeholder geometry.
- The finale is mostly concentric bordered circles.
- The home page is a sparse title card rather than a cinematic arrival.
- Successful player authentication reloads the page, interrupting visual continuity.
- Current large components are too monolithic for clean animation ownership.
- CSS has accumulated many scene rules in one global file.

Do not preserve these weaknesses for compatibility. Preserve behavior and semantics while rebuilding presentation and architecture.



4. Dependency installation and migration

4.1 Use official packages

Install the current stable compatible versions after checking official documentation and npm metadata. Pin the exact installed versions in `package.json` and lockfile, matching the repository's reproducibility style.

Expected package work:

```
npm uninstall framer-motion
npm install motion gsap @gsap/react page-flip @rive-app/react-webgl2 lottie-web
```

Then:

- Replace all imports from `framer-motion` with imports from `motion/react`.
- Remove direct `motion-dom` and `motion-utils` dependencies unless the project directly imports them for a documented reason. Let `motion` own compatible transitive versions where possible.
- Use GSAP 3.13 or later so the relevant official plugins are available through normal npm installation.
- Register only the GSAP plugins actually used.
- Centralize GSAP registration in one client-safe module so tree shaking does not remove plugins and registration is not scattered across components.
- Use `@gsap/react` and `useGSAP` for React lifecycle integration.
- Use the core `page-flip` package behind a custom React wrapper. Do not depend on an unverified React wrapper unless it is tested with the current React version and provides a clear advantage.
- Use the recommended Rive React WebGL2 runtime.
- Use official `lottie-web` through a custom React wrapper with explicit lifecycle control.

4.2 Do not blindly install and import globally

- Motion may be globally available where used throughout the UI.
- GSAP must be imported only in client animation modules and scene builders.
- StPageFlip must be dynamically imported only on routes and components that display a book.
- Rive must be dynamically loaded only around Rive objects.
- Lottie must be dynamically loaded only when an effect is visible or imminently needed.
- Do not put all animation libraries into the initial landing bundle.
- Do not load the development showcase in production bundles where avoidable.

4.3 Build must remain clean

After dependency migration:

- No `framer-motion` imports remain.
- No duplicated incompatible Motion packages remain.
- No hydration warnings occur.
- No library accesses `window`, `document`, canvas, WebGL, or layout during server rendering.
- Client-only wrappers are clearly marked.
- Dynamic imports have proper loading and fallback states.



5. Animation architecture

Create a dedicated animation system instead of leaving orchestration inside `PlayerExperience.tsx`, `Quartermaster.tsx`, or global CSS.

A reasonable structure is:

```
src/
  animation/
    core/
```

```

gsap-client.ts
animation-types.ts
ownership.ts
visibility.ts
quality.ts
asset-registry.ts
director/
  AnimationDirector.ts
  AnimationProvider.tsx
  useAnimationDirector.ts
  scene-registry.ts
  operation-sync.ts
scenes/
  arrival.scene.ts
  player-access.scene.ts
  journal-open.scene.ts
  chapter-release.scene.ts
  map-reveal.scene.ts
  artifact-award.scene.ts
  quest-discovery.scene.ts
  log-entry.scene.ts
  finale-tease.scene.ts
  quartermaster-login.scene.ts
  quartermaster-actions.scene.ts
  pause-resume.scene.ts
  undo.scene.ts
motion/
  variants.ts
  transitions.ts
  reduced-motion.ts
assets/
  rive-contracts.ts
  lottie-contracts.ts
  svg-contracts.ts
components/
  animation/
    RiveStatefulObject.tsx
    LottieEffect.tsx
    PageFlipBook.tsx
    CinematicOverlay.tsx
    AnimationBoundary.tsx
    AnimationControls.tsx
    AssetFallback.tsx
dev/
  AnimationTestButton.tsx
  AnimationShowcase.tsx
  AnimationTrailer.tsx
  AnimationMetrics.tsx

```

Adapt names to the existing conventions, but preserve the separation of concerns.

5.1 AnimationDirector

Create a typed `AnimationDirector` that owns active cinematic scenes.

It must support:

- `play(sceneName, context)`
- `pause()`
- `resume()`
- `seek(labelOrProgress)`
- `setSpeed(multiplier)`
- `skip()`
- `reverse()` where semantically valid
- `cancel(reason)`
- `kill()`
- `isPlaying`
- current scene name

- current label or stage
- progress from 0 to 1
- full, gentle, and reduced-motion modes
- prevention or controlled queuing of conflicting scenes
- cleanup on unmount, route change, logout, and document visibility changes

Do not expose internal GSAP implementation details throughout the application. Components should ask the director to play named scenes.

5.2 Scene definitions

Each cinematic scene must be a real GSAP timeline or a controller that contains a real timeline.

Each scene must:

- Use named GSAP labels.
- Use overlapping animations rather than serial delays wherever physically appropriate.
- Resolve from actual completion events.
- Return a controller or promise that supports cancellation.
- Revert styles and split text on cleanup.
- Pause ambient animation when the document is hidden.
- Resume safely without duplicating events.
- Provide a full, gentle, and reduced-motion implementation.
- Contain no authoritative business logic.
- Accept sanitized display data through typed context.

Examples of labels:

```
scene-start
attention
physical-open
await-server
success-branch
failure-branch
content-readable
interaction-restored
scene-complete
```

5.3 Server-operation synchronization

Replace the old pattern where a timer sequence simply runs beside a request.

For server-backed actions:

1. Start the real server request immediately when the user confirms.
2. Start the physical opening segment of the scene.
3. Advance to a named `await-server` synchronization label.
4. If the server response is still pending, hold at that label with an intentional living idle state, not a frozen blank screen.
5. On success, inject or resume the success branch.
6. On failure, play a physically meaningful failure or reversal branch.
7. Never update authoritative visual state before success.
8. Never reveal protected content while waiting.
9. If the user selects Skip while the server is pending, accelerate or skip the nonessential opening, then remain at the safe synchronization point.
10. After success, Skip may jump to the final truthful visual state.

Use GSAP timeline labels and precise pause/resume behavior rather than manual timeout guesses.

5.4 Animation ownership

Document and enforce the rule:

> One engine owns the transform, opacity, filter, clip-path, and geometry of an element at a time.

Add a development-only ownership helper. Components may declare an owner through a data attribute or registry, such as:

```
<div data-animation-owner="st-page-flip" />
```

In development, warn when a scene attempts to assign conflicting ownership.

Examples:

- StPageFlip owns page curl, page rotation, page shadow, and page geometry.
- GSAP may animate the whole book wrapper, light, seal, and page content after a flip.
- Motion owns the navigation button that selects the Journal.
- Rive owns the internal layers of a journal clasp canvas.
- Lottie owns its own SVG or canvas effect container.

Do not let Motion and GSAP both animate **transform** on the same target. If composition is needed, use nested wrappers with separate ownership.

5.5 Full, gentle, and reduced modes

Preserve the existing three-level user control.

Full

- Complete camera movement
- Page curl
- Ambient loops
- Particle effects
- Long-form writing where narratively justified
- Detailed Rive state transitions
- Full GSAP timeline timing

Gentle

- Same narrative order
- Smaller travel distances
- Shorter camera pushes
- Lower particle counts
- Fewer continuous loops
- Reduced parallax
- Faster text reveals
- Page turns remain physical but calmer

Reduced

- No sweeping camera moves
- No continuous ambient loops
- No dragged page curl requirement
- Use short opacity transitions and static end-state poses
- Reveal text by paragraph or line groups quickly
- Preserve focus and content order
- Preserve all interaction and state meaning
- Rive should display stable state poses and avoid idle loops
- Lottie should show a representative still frame or not mount
- StPageFlip should use immediate page changes or a short crossfade fallback

Use both the user's in-app preference and **prefers-reduced-motion**. The more restrictive setting wins unless the existing product behavior defines otherwise.



6. Library-specific implementation rules

6.1 GSAP

Use GSAP for cinematic orchestration and advanced text, SVG, and state-transition effects.

Use these plugins where justified:

- `SplitText`
- `DrawSVGPlugin`
- `MorphSVGPlugin`
- `MotionPathPlugin`
- `Flip`
- `CustomEase`
- `TextPlugin` only for short typewriter-like operational text, not long story prose

Requirements:

- Register plugins centrally.
- Use `useGSAP` or `gsap.context` with cleanup.
- Revert every context on unmount.
- Revert `SplitText` instances after scenes or when responsive re-splitting requires it.
- For line-based `SplitText`, use `autoSplit` and create the animation inside `onSplit`.
- Keep screen-reader semantics intact.
- Do not split thousands of characters unnecessarily.
- Use line and word masks for long prose.
- Use character-level animation only for short headings, signatures, or one dramatic line.
- Use `DrawSVG` for actual SVG stroke paths.
- Use `MotionPath` for quills, compass needles, route particles, and object paths.
- Use GSAP `Flip` for moving an artifact from a reveal scene into its altar slot or moving notes between layouts.
- Do not use GSAP `Flip` for 3D page curl. `StPageFlip` owns pages.

6.2 StPageFlip

Create a custom `PageFlipBook` client component around `page-flip`.

Requirements:

- Initialize only after page elements exist.
- Use HTML pages so text stays accessible and selectable.
- Use `data-density="hard"` for covers and appropriate hard dividers.
- Use soft pages for chapter content.
- Use `loadFromHTML` for first mount.
- Use `updateFromHtml` when the page collection changes.
- Listen for `flip`, `changeOrientation`, and `changeState` events.
- Expose imperative controls through a typed ref:
 - `next`
 - `previous`
 - `turn to page`
 - `flip to page`
 - `current page`
 - `page count`
 - `current orientation`
- Call `destroy()` on unmount and before reinitialization.
- Preserve keyboard page navigation.
- Provide visible previous and next controls.
- Provide direct chapter navigation that turns to the correct page.
- Do not require drag gestures.
- Avoid trapping mobile scrolling inside a page unintentionally.
- Test `mobileScrollSupport` behavior.
- Use portrait mode on narrow screens and landscape spread on larger screens.
- Preserve focus when pages change.

- Do not duplicate chapter content in accessible navigation in a way that causes screen readers to read both copies. Initial settings should be tuned through testing, not copied blindly. A likely starting point is:

```
{
  width: 560,
  height: 760,
  size: "stretch",
  minWidth: 300,
  maxWidth: 720,
  minHeight: 420,
  maxHeight: 960,
  showCover: true,
  usePortrait: true,
  autoSize: true,
  drawShadow: true,
  maxShadowOpacity: 0.45,
  flippingTime: 1100,
  swipeDistance: 24,
  mobileScrollSupport: true
}
```

Adjust from observed desktop and phone behavior. Full mode may use a longer physical flip. Gentle mode may shorten it. Reduced mode should bypass physical curl.

6.3 Motion

Migrate to the `motion` package and use `motion/react`.

Motion owns ordinary application interactions:

- Hover
- Tap
- Focus
- Buttons
- Navigation
- Layout transitions
- Modals
- Dialogs
- Lists
- Cards
- Unseen badges
- Objective dock
- Preferences
- Artifact inspection shell
- Responsive rearrangement
- Mobile sheets

Requirements:

- Create shared variants and transitions.
- Use `MotionConfig` for reduced-motion and shared transition policy.
- Use `LazyMotion` if it measurably reduces bundle size without creating complexity.
- Use `AnimatePresence` for real exits, not only one small toolbar.
- Use keyed section views.
- Use `mode="wait"` where incoming content must wait for the outgoing physical view.
- Use `layout` and `layoutId` for shared elements.
- Use `useReducedMotion` in addition to product preference.
- Avoid layout thrashing.
- Use nested wrappers when GSAP must animate a parent while Motion animates a child.

6.4 Rive

Use `@rive-app/react-webgl2` in isolated wrapper components.

Rive is for stateful hero objects:

- Invitation seal
- Journal clasp
- Voyage compass
- Map compass rose
- Artifact objects or artifact-family rigs
- Finale mechanism
- Campaign status instrument
- Connection lantern
- Quest cipher wheel

Requirements:

- Isolate each `useRive` call in a stable wrapper.
- Use local `.riv` assets in `public/animations/rive`.
- Do not load random remote community files in production.
- Disable automatic Rive asset CDN loading unless a documented local-asset reason requires it.
- Use a typed asset contract that records:
 - `file path`
 - `artboard name`
 - `state machine name`
 - `input or data-binding names`
 - `expected states`
 - `fallback component`
- Use `useRiveFile` when the same parsed file can be reused safely across multiple instances.
- Clean up Rive instances and release WebGL resources on unmount.
- Stop rendering when offscreen or document-hidden where supported.
- Show an accessible HTML fallback if WebGL fails or the asset cannot load.
- Never make essential text available only inside canvas.
- Keep application state in React or the domain model, not inside the Rive file.

Important truth about asset authoring

Do not fake Rive implementation.

A valid `.riv` file is a binary design asset normally authored in the Rive editor. If the repository does not contain valid original Rive files and the current environment cannot author them correctly:

1. Implement the full Rive runtime, wrappers, typed state-machine contracts, loading, error handling, cleanup, and test coverage.
2. Use a clearly license-approved local sample only in the development animation showcase to prove runtime and state-machine control.
3. Record the source and license.
4. Keep production-facing objects on polished original SVG/GSAP or Motion fallbacks until original `.riv` files are supplied.
5. Create exact Rive authoring specifications in `docs/animation/rive-assets.md` so a designer can produce files without guessing names or states.
6. Do not claim production Rive objects exist if they do not.

If valid original `.riv` assets are available, integrate them fully and remove temporary sample usage.

6.5 Lottie

Use official `lottie-web` through a custom `LottieEffect` wrapper.

Lottie is for pre-authored atmosphere and effects:

- Waves
- Fog
- Dust
- Ink bloom
- Wax debris
- Page dust
- Map fog
- Artifact particles

- Celestial particles
- Underwater caustics
- Loading flourishes

Requirements:

- Support SVG and canvas renderer selection per asset.
- Default to SVG for crisp decorative shapes.
- Use canvas for particle-heavy effects only when profiling justifies it.
- Use local JSON assets in `public/animations/lottie`.
- Validate JSON assets at build or test time.
- Call `destroy()` on unmount.
- Pause when offscreen.
- Pause or freeze when the document is hidden.
- Do not mount continuous loops in reduced mode.
- Expose typed controls:
 - `play`
 - `pause`
 - `stop`
 - `go to frame`
 - `play segment`
 - `speed`
 - `direction`
 - `destroy`
- Support `complete`, `loopComplete`, `data_ready`, and `data_failed` events.
- Provide a static fallback image or CSS shape.
- Do not depend on remote runtime assets.

If sophisticated Bodymovin exports do not exist, Codex may create simple original shape-layer Lottie JSON for ambient loops and validate it in the animation showcase. Do not hand-author a giant invalid JSON file and declare victory.



7. Visual asset and content system

Create organized asset folders:

```
public/
  animations/
    rive/
    lottie/
    svg/
    stills/
  textures/
  illustrations/
    journal/
    chart/
    artifacts/
    quests/
    finale/
```

Create an asset manifest in TypeScript or JSON containing:

- key
- type
- local path
- owner library
- intended pages
- full/gentle/reduced behavior
- preload priority
- license or provenance reference
- fallback path

- expected dimensions or viewBox

Update asset-license documentation.

Use only original or properly licensed assets.

Where real finished art is unavailable, create high-quality original procedural SVG and CSS artwork as a fallback. Do not leave generic diamonds, empty bordered circles, or raw library demo art in the final production UI.



8. Page-by-page implementation

8.1 Harbor landing page

The current landing page is too sparse. Rebuild it as a cinematic arrival.

Add visual content

Create layered scene content:

1. Star field
2. Moon glow
3. Distant clouds
4. Horizon
5. Distant ship silhouette
6. Moonlit ocean
7. Rolling fog
8. Foreground dock, ropes, timbers, or cabin silhouette
9. Hanging lantern
10. Central interactive emblem
11. Nautical border that appears after the scene settles

The environment must remain original nautical fantasy, not copied Sea of Thieves art.

Library assignments

Lottie

- Slow wave loops
- Rolling horizon fog
- Cloud drift
- Lantern particles
- Occasional spray
- Moonlight ripple accents

Use long loops with unobvious repetition.

Rive

Use a central emblem, compass, or sealed medallion with:

- idle
- hover
- pressed
- invitation-detected
- opening-voyage
- reduced static pose

If a production Rive asset is unavailable, use the documented fallback and validate Rive in the development showcase.

GSAP

Build the first-arrival timeline:

1. Begin almost black.
2. Reveal faint ocean and horizon.
3. Separate fog layers.
4. Bring moonlight onto the water.
5. Reveal ship silhouette.
6. Draw or write the main title.
7. Wake the emblem.
8. Reveal the paragraph line by line.
9. Raise the call-to-action.
10. Settle into ambient mode.

Retain the current narrative concepts such as dark sea, tide arrival, title written, voyage materialization, and content arrival, but implement them as real labels and timelines.

Motion

- Button hover and press
- Focus ring transitions
- Small pointer parallax
- Responsive layout shifts
- Shared transition into the invitation scene

Acceptance criteria

- The first arrival feels intentional and layered.
- Same-session reentry is shorter.
- Replay is available.
- Skip is available after the opening establishes context.
- Reduced mode shows the complete scene quickly without large motion.
- No giant blank fade lasting several seconds.



8.2 Player invitation and access gate

Transform the access gate into a physical invitation scene.

Add visual content

- Envelope or folded parchment
- Wax seal
- Ribbon or cord
- Recipient line
- Phrase field integrated into the paper
- Moonlit tabletop or chart-room desk
- Quill, compass, coin, or key props
- Lantern illumination
- Reflected tide light

Failure behavior

- Seal resists.
- Brass or clasp twitches but stays locked.
- Entered ink subtly recedes.
- Card darkens briefly.
- Error appears as a marginal captain's note.
- Focus returns correctly.

- No screen shake large enough to be annoying.

Success behavior

1. Ink under the phrase activates.
2. Seal warms.
3. Seal deforms.
4. Crack paths spread.
5. Wax breaks or releases.
6. Ribbon loosens.
7. Invitation unfolds.
8. Camera approaches the parchment.
9. Parchment becomes the journal cover or first page.
10. Player experience mounts without an abrupt full-page reload.

Replace the current unconditional `location.reload()` success path with a secure transition that revalidates or navigates through the existing authenticated route without exposing content early.

Library assignments

- Rive: seal and clasp state machine
- GSAP: request-synchronized success and failure timeline
- Lottie: dust, wax fragments, edge fog, ink bloom
- Motion: form interactions, validation, focus, responsive card movement

Acceptance criteria

- Authentication remains secure.
- Success animation waits for server success.
- Failure reverses honestly.
- Skip never bypasses access.
- No protected payload is requested before authorization.



8.3 Main player companion shell

Make the six sections feel like one physical workspace rather than unrelated pages.

Add content

- Journal centered on a table or story workspace
- Chart sliding or unfolding from one side
- Relic altar or cabinet from the other
- Side-quest ledger as a secondary physical book
- Ship's Log as a separate logbook or engraved timeline
- Finale mechanism visibly dormant beneath or behind the workspace
- Brass instruments for navigation
- Physical unread indicators
- Current objective as a pinned note, ribbon, or captain's tag rather than a generic software dock

Motion ownership

Use Motion for:

- Shared navigation indicator
- Section enter and exit
- Objective expansion
- Preferences
- Unseen badges

- Connection state
- Fullscreen state
- Mobile bottom navigation
- Artifact inspection shell
- Responsive rearrangement

Wrap main sections in keyed **AnimatePresence**. Use `mode="wait"` where physical continuity requires the old view to leave first.

GSAP ownership

Use GSAP only for first-visit or story-critical physical transformations:

- Journal closing before chart focus
- Chart unfolding
- Altar curtain parting
- Ledger placement
- Finale chamber opening

Routine repeated navigation should remain faster through Motion.

Acceptance criteria

- Back and forward browser navigation remains correct.
- Query parameter section deep links remain correct.
- Focus moves to the new section heading.
- No section starts a duplicate SSE or polling connection.
- Mobile transitions do not cover essential controls.



8.4 Journal

The Journal is the centerpiece. Rebuild it as a convincing physical book.

Add physical book content

- Hard front and back covers
- Embossed title
- Metal corners
- Spine stitching
- Uneven page edges
- Visible page stack
- Endpapers
- Title page
- Ownership or dedication page using generic development copy
- Chapter divider pages
- Sealed blank pages
- Captain handwriting
- Marginalia
- Sketches
- Crossed-out corrections
- Ink stains
- Water spots
- Folded corners
- Inserted notes
- Ribbons and bookmarks
- Page numbers
- Chapter tabs
- Hidden visual details safe for public development content

Page model

Do not force one chapter into one giant page.

Create a typed page model capable of representing:

1. Cover
2. Endpaper
3. Title
4. Chapter divider
5. Narrative page
6. Objective and riddle page
7. Hint page
8. Annotation page
9. Related map or artifact page
10. Locked page
11. Back matter

Pages must derive only from the sanitized public snapshot.

StPageFlip

Own:

- Cover opening
- Page curling
- Page shadows
- Desktop spread
- Mobile portrait page
- Programmatic turning to active chapter
- Manual next and previous
- Dragging where supported

GSAP

After StPageFlip completes a page turn:

- Write chapter heading
- Reveal narrative by line masks
- Ink riddle with distinct rhythm
- Stamp or pin objective
- Draw small illustrations
- Dry wet ink
- Move quill along an approximate route
- Reveal captain signature
- Animate corrections or marginal notes

Live-writing rules

Long dynamic prose

- Use SplitText lines and words.
- Use masks.
- Use subtle timing variation.
- Use a wet-ink overlay that darkens and dries.
- Do not type every character individually.

Short headings and signatures

- Use SVG paths and DrawSVG for true stroke drawing.
- Character splitting is acceptable for a short title when path art does not exist.

Accessibility

- Preserve one readable semantic copy.
- Do not make a screen reader read every letter separately.
- Test font loading and responsive re-splitting.

Rive

Use for:

- Journal clasp
- Locked-chapter seal
- Cover compass
- Stateful bookmark

Lottie

Use for:

- Paper dust
- Ink splatter
- Moving paper fibers
- Water stain
- Seal debris

Motion

Use for:

- Chapter index
- Hint panels
- Annotation cards
- Page controls
- Mobile controls
- Expanded detail panels

Acceptance criteria

- Physical page turn is clearly more than a small rotate transform.
- Content remains selectable.
- Chapter page index remains stable after snapshot updates.
- Current page survives safe re-renders.
- StPageFlip is destroyed cleanly.
- Keyboard users can navigate every page.
- Reduced mode does not force page curl.



8.5 Voyage Chart

Rebuild the chart into an illustrated interactive map.

Add content

- Original illustrated map background
- Accurate project island illustrations where available and permitted
- Region boundaries
- Compass rose
- Rhumb lines
- Grid coordinates
- Fold marks
- Edge wear

- Sea-monster or nautical sketch details
- Discovered and undiscovered regions
- Fog-of-war masks
- Route paths
- Ship token
- Destination markers
- Traveled history
- Captain notes
- Side-quest symbols
- Underwater-location treatment

Do not use copyrighted game map art.

GSAP

- Draw routes with DrawSVG.
- Remove or morph fog masks.
- Draw destination rings.
- Stamp markers.
- Move ship token with MotionPath.
- Turn compass needle.
- Reveal region highlight.
- Write coordinate notes.
- Reverse route drawing for undo.

Rive

- Compass rose
- Ship token
- Destination marker
- Storm marker
- Underwater marker
- Optional mystery marker

Lottie

- Fog
- Water shimmer
- Cloud shadows
- Storm movement
- Underwater bubbles or caustics
- Discovery sparkle

Motion

- Pan and zoom interface
- Marker hover cards
- Selection
- Text alternative
- Filters
- Mobile map sheet
- Marker layout

Acceptance criteria

- Route animation follows actual route data.
- Hidden coordinates remain absent.
- Text alternative remains complete.
- Pan and zoom are keyboard accessible.
- Undo retracts the correct path rather than hiding the whole map.



8.6 Treasure Altar

Replace abstract placeholder buttons with a physical collection altar or cabinet.

Add content

- Dark wood or carved stone
- Velvet backing
- Brass slots
- Empty silhouettes
- Chains or mounting hardware
- Engraved labels
- Artifact connection lines
- Dust and light shafts
- Artifact illustrations
- Material variation
- Central incomplete assembly outline
- Inspection pedestal

Rive

Major artifacts should support state concepts:

- unknown
- silhouette
- discovered
- awarded
- inspected
- connected
- active
- completed

Use reusable rigs where possible instead of one huge runtime per trivial object.

GSAP artifact ceremony

1. Altar darkens.
2. Shadow or curtain closes.
3. Silhouette appears.
4. Light sweeps.
5. Object emerges.
6. Object rotates toward player.
7. Name writes.
8. Description appears.
9. GSAP Flip moves object into permanent altar slot.
10. Related artifacts pulse and connection path draws.

Lottie

- Dust
- Velvet particles
- Glints
- Smoke
- Trails
- Light rays

Motion

- Artifact selection

- Hover and tap
- Inspection overlay
- Description expansion
- Mobile inspection
- Connection details

Acceptance criteria

- Artifact state remains server-driven.
- Unknown artifacts do not reveal names.
- Shared-element motion correctly returns to the same altar slot.
- Essential information is not canvas-only.



8.7 Side-Quest Ledger

Rebuild the current card grid as a physical secondary ledger while retaining efficient access.

Add content

- Smaller rough leather book
- Straps
- Loose notes
- Folded correspondence
- Pinned scraps
- Red thread
- Ciphers
- Checkboxes
- Stamps
- Wax pins
- Reward pockets
- Secret flaps
- Rumor, Active, and Complete dividers

StPageFlip

Use for:

- Ledger cover
- Section dividers
- Quest spreads
- Rumor pages
- Completed pages

GSAP

- New note slides from envelope
- Red thread draws between clues
- Objective handwriting
- Completion stamp
- Reward pocket opens
- Cipher appears
- Checkmarks ink in
- Torn paper unfolds

Motion

- Filtering
- Objective completion

- Checklist layout
- Tabs
- Stacked notes
- Mobile details

Rive

- Cipher wheel
- Quest compass token
- Stateful wax pin
- Reward medallion

Lottie

- Paper scraps settling
- Stamp burst
- Dust
- Candle smoke

Acceptance criteria

- Optional quests never block the main voyage.
- Hidden quests remain absent.
- Rumor-safe teaser rules remain intact.
- Quest page updates do not reset the book unexpectedly.



8.8 Ship's Log

Turn the activity feed into a captain's log while retaining searchability and fast navigation.

Add content

- Dated logbook entries
- Handwriting
- Weather symbols
- Moon phase
- Location notation
- Small illustrations
- Ink color by category
- Important-event stamps
- Day dividers
- Timeline ribbon
- Captain commentary
- Page-end flourishes

Motion

Primary engine for:

- New entry insertion
- List reflow
- Unseen indicator
- Expansion
- Date grouping
- Filter transitions
- Navigation links

GSAP

Use for important first-time entries:

1. Date writes.
2. Heading appears.
3. Body inks in.
4. Symbol stamps.
5. Ink dries.
6. Page settles.

Do not play a long ceremony for every trivial event.

Lottie

- Ink blot
- Tiny weather animations
- Page dust
- Stamp particles
- Moon shimmer

Rive

- Category emblems
- Voyage status
- Moon-phase instrument
- Captain seal

Optional StPageFlip mode

A secondary **Read as logbook** mode may page through dated entries. The default timeline must remain efficient and searchable.



8.9 Finale chamber

Replace the current bordered circles with a dormant ancient celestial mechanism.

Add content

- Layered mechanical rings
- Compass arms
- Star positions
- Artifact sockets
- Brass, stone, and moonlit glass
- Chains or locks
- Engraved runes
- Central sealed core
- Requirement indicators integrated into the machine
- Constellation background
- Light paths between completed requirements
- Artifact silhouettes around perimeter
- Locked inner chamber
- Hidden unreadable geometry

Do not add real final surprise content while repository visibility is unsafe.

Rive

The finale mechanism is the strongest Rive use case.

Design the integration contract for:

- finale state
- requirements completed
- requirements total
- optional completed
- artifact count
- hover
- inspect
- activating
- unlocked
- complete

Expected visual states:

- dormant
- teased
- sealed
- partial
- ready
- unlocking
- unlocked
- complete

If a real original `.riv` file is unavailable, implement the contract and a polished SVG/GSAP fallback. Demonstrate Rive state-machine control in the development showcase using a licensed local sample.

GSAP

Orchestrate:

1. Workspace darkens.
2. Interface recedes.
3. Camera approaches.
4. Fog gathers.
5. Mechanism wakes.
6. Requirements activate sequentially.
7. Connection paths draw.
8. Title writes.
9. Core begins unlocking.
10. Light fills screen.
11. Final safe content appears.

Lottie

- Celestial dust
- Fog
- Light rays
- Constellation particles
- Energy trails
- Debris
- Water or stars behind mechanism

Motion

- Requirement list
- Inspection controls
- Progress counters
- Optional markers
- Exit navigation

Acceptance criteria

- Current Phase Two structural limits remain enforced.
- Unsupported final states are not enabled accidentally.
- Requirement progress is truthful.
- Reduced mode offers a static but meaningful mechanism state.



8.10 Artifact inspection

Rebuild the artifact modal as a physical inspection experience.

Add content

- Dimmed altar remains visible
- Inspection pedestal
- Large object illustration
- Pointer-follow light
- Material response
- Rotate or tilt interaction
- Engraved metadata plate
- Foldout story card
- Connection diagram

Motion

- Shared-element transition from slot to pedestal
- Overlay entry and exit
- Pointer tilt
- Swipe close on mobile
- Metadata expansion

Rive

- Artifact object
- Hover
- Inspection
- Connected state
- Material shimmer
- Hidden safe detail

GSAP

- First inspection ceremony
- Engraving reveal
- Connection paths
- Return sequence

Lottie

- Dust
- Glint
- Smoke
- Small particles



8.11 Chapter-release ceremony

Rebuild the existing stage sequence as the flagship cinematic.

Omen

- Ocean sound recedes.
- Lantern flickers.
- Navigation becomes secondary.
- Shadow crosses workspace.
- Bookmark reacts.

Libraries: GSAP, Lottie, Rive.

Attention

- Camera moves toward Journal.
- Side panels retreat.
- Book centers.
- Objective note folds away.
- Clasp awakens.

Libraries: GSAP, Motion, Rive.

Seal

- Seal warms.
- Wax deforms.
- Crack paths draw.
- Fragments separate.
- Dust and crumbs fall.
- Sound peaks at break.

Libraries: Rive, GSAP DrawSVG or MorphSVG, Lottie.

Parchment

- StPageFlip turns to correct spread.
- Edges flutter.
- Light crosses pages.
- Page settles.

Libraries: StPageFlip, GSAP, Lottie.

Heading

- Chapter number appears.
- Title writes.
- Illustration draws.
- Captain mark appears.

Libraries: GSAP SplitText and DrawSVG.

Story

- Narrative reveals line by line.
- Wet ink darkens and dries.
- Quill follows path.
- Marginal sketch appears.

Libraries: GSAP, MotionPath, Lottie.

Objective

- Objective card unfolds or pins in.
- Heading stamps.
- Text writes.
- Compass points.

Libraries: GSAP, Rive, Motion.

Riddle

- Surrounding page quiets.
- Riddle appears with distinct rhythm.
- Important symbols respond.
- Optional cipher layer appears.

Libraries: GSAP SplitText, Rive.

Map

- Chart edge appears.
- Fog parts.
- Marker stamps.
- Route draws.
- Compass turns.
- Ship token begins course.

Libraries: GSAP DrawSVG and MotionPath, Rive, Lottie.

Active

- Camera returns.
- Navigation returns.
- Objective note reappears.
- Ambient loops resume.
- Unseen indicators activate.
- Player control returns.

Libraries: GSAP and Motion.

Timing

Full mode may take roughly 7 to 12 seconds if the sequence contains meaningful overlapping action. Gentle should be shorter. Reduced should complete quickly. Do not create long dead air.



8.12 Quartermaster login

Create a full chart-room entrance.

Add content

- Cabin door
- Keyhole
- Hanging lantern
- Door shadow
- Captain ledger
- Moving light
- Lock mechanism
- Glimpse of chart room

Rive

- Lock and key mechanism
- Door emblem
- Authentication state
- Lantern response

GSAP

- Key turns
- Bolts retract
- Door opens
- Light sweeps
- Ledger rises
- Dashboard assembles
- Failure reverses mechanism

Lottie

- Door dust
- Lantern motes
- Smoke
- Light rays
- Brass sparks

Motion

- Form
- Errors
- Button
- Input focus
- Dashboard mount



8.13 Quartermaster dashboard

Keep the GM interface efficient while making it physically cohesive.

Add content

- Captain's desk layout
- Campaign status instruments
- Player connection lamp
- Chapter seal rack
- Preview parchment
- Action ledger
- Event logbook
- Artifact drawer
- Side-quest folder
- Finale indicator
- Small map overview

Motion

Primary engine:

- Cards
- Data updates
- Action selection

- Confirmation
- Event insertion
- Toasts
- Inventory reflow
- Responsive layout
- Busy states

GSAP

Compact action ceremonies for:

- Prepare
- Release
- Solve
- Award artifact
- Reveal map
- Reveal route
- Connect artifacts
- Discover quest
- Complete quest
- Tease finale
- Undo
- Pause
- Resume

GM ceremonies should generally be 1.5 to 4 seconds. The player may receive the longer cinematic.

Rive

- Connection lamp
- Campaign instrument
- Chapter seal
- Finale status
- Inventory emblem

Lottie

- Loading ledger
- Success flourish
- Error effect
- Ink writing accent
- Progress indicator

StPageFlip

Optional appropriate uses:

- Event logbook
- Player preview pages
- Snapshot review

Do not make primary progression controls depend on flipping pages.



8.14 Quartermaster action overlays

Delete the one-size-fits-all stage-name overlay. Create distinct action visuals.

Prepare chapter

- Blank page aligns.

- Seal presses.
- Ready mark appears.

Release chapter

- Seal breaks.
- Page opens.
- Light spills.
- Ink starts moving.

Mark solved

- Stamp rises.
- Page compresses.
- Imperfect solved mark appears.

Reveal map

- Fog chart appears.
- Compass turns.
- Marker stamps.
- Route draws.

Award artifact

- Velvet darkness.
- Silhouette.
- Light sweep.
- Object enters slot.

Connect artifacts

- Both objects illuminate.
- Fine line draws.
- Mechanism responds.

Undo

- Ink returns to quill.
- Stamp lifts.
- Route retracts.
- Artifact returns to prior truthful state.

Pause

- Wind falls.
- Lantern dims.
- Compass slows.
- Pause stamp appears.

Resume

- Lantern kindles.
- Compass resumes.
- Ocean ambience returns.
- Pause mark lifts.



9. Audio synchronization

Preserve user gesture requirements for Web Audio.

Create a typed audio-cue layer that can be called from GSAP labels without putting audio logic into every scene.

Possible cues:

- ocean rise
- distant wood creak
- lantern ignition
- brass latch
- seal pressure
- wax crack
- page turn
- paper flutter
- quill scratch
- stamp impact
- compass click
- map scratch
- artifact chime
- mechanism hum
- pause wind-down
- undo reverse texture

Requirements:

- Respect mute and master volume.
- Do not autoplay before user interaction.
- Stop or fade cues on scene cancellation.
- Avoid copyrighted game audio.
- Use original procedural audio or properly licensed local assets.
- Record asset provenance.
- Provide reduced sensory mode if needed.
- Do not let audio completion block authoritative state.



10. Development-only animation showcase and trailer

This is mandatory.

Create a development-only button labeled exactly:

TEST ANIMATIONS

10.1 Button placement

- Visible only in development or when an explicit non-production environment flag enables it.
- Place it unobtrusively in a fixed corner of the player interface and Quartermaster interface.
- Do not render it in production.
- Give it an accessible name and visible focus state.
- Avoid overlapping mobile navigation and safe-area insets.

10.2 Development route

Create a dedicated route such as:

/dev/animations

Requirements:

- Return 404 or otherwise be inaccessible in production.

- Do not require or mutate real campaign state.
- Do not expose secrets.
- Use deterministic local development data.
- Code-split the route.

The **TEST ANIMATIONS** button should open this route or a full-screen showcase overlay.

10.3 Animation Showcase features

The showcase must include:

- Primary **PLAY TRAILER** button
- Scene selector
- Library selector
- Full, gentle, and reduced modes
- Play
- Pause
- Resume
- Restart
- Skip
- Reverse where supported
- Timeline scrubber
- Current scene label
- Progress percentage
- Playback speed: 0.25x, 0.5x, 1x, 1.5x, 2x
- Fullscreen button
- Asset-loading status
- Fallback status
- Error log
- Basic performance metrics
- Keyboard shortcuts
- Reset button

Suggested keyboard shortcuts:

- Space: play or pause
- R: restart
- S: skip
- Left and Right: seek or previous/next scene when safe
- Escape: close or exit fullscreen

10.4 Individual scene demos

Include non-destructive demos for:

1. First arrival
2. Same-session reentry
3. Player access listening
4. Player access success
5. Player access failure
6. Quartermaster login success
7. Quartermaster login failure
8. Journal cover opening
9. Manual page flip
10. Programmatic page flip
11. Live chapter-heading writing
12. Long prose ink reveal
13. Seal deformation and cracking
14. Chapter-release full ceremony
15. Map fog reveal
16. Route drawing

17. Marker stamp
18. Ship token MotionPath
19. Artifact silhouette reveal
20. Artifact award
21. Artifact inspection
22. Artifact connection
23. Quest discovery
24. Quest completion stamp
25. New Ship's Log entry
26. Finale tease
27. Finale requirement activation
28. Pause
29. Resume
30. Undo reversal
31. Motion navigation transitions
32. Rive state-machine inputs
33. Lottie play, pause, segment, speed, and destroy behavior
34. StPageFlip portrait and landscape behavior
35. Reduced-motion equivalents
36. Error and fallback states

10.5 Cinematic trailer

Create a polished non-destructive trailer, approximately 45 to 90 seconds in full mode. It should demonstrate the system as a coherent miniature voyage.

Suggested trailer flow:

1. Moonlit ocean arrival
2. Title writing
3. Invitation seal recognizes phrase
4. Journal opens
5. Page turns
6. Chapter heading writes
7. Riddle appears
8. Chart fog parts
9. Route draws
10. Artifact emerges
11. Artifact enters altar
12. Side-quest note unfolds
13. Log entry writes
14. Finale mechanism briefly wakes
15. Return to title card with library and scene summary

The trailer must:

- Use mock safe content only.
- Never call progression APIs.
- Never alter database state.
- Be replayable.
- Be skippable.
- Run in all three motion modes.
- Handle missing Rive assets with visible fallback labels rather than crashing.
- Demonstrate each installed library clearly.

10.6 Performance panel

Display development metrics such as:

- Current FPS estimate
- Current scene
- Active GSAP timeline count if available
- Mounted Rive instances
- Mounted Lottie instances
- PageFlip instance status
- Document visibility state
- Motion mode
- Asset load failures
- Long-task count observed through PerformanceObserver where supported

Do not turn this into production telemetry.



11. Component decomposition

Break large monolithic components into maintainable physical sections before adding extensive animation.

Likely player components:

- `PlayerShell`
- `CompanionHeader`
- `CompanionNavigation`
- `ObjectiveNote`
- `JournalWorkspace`
- `VoyageChart`
- `TreasureAltar`
- `SideQuestLedger`
- `ShipsLog`
- `FinaleChamber`
- `ArtifactInspection`
- `PlayerPreferences`
- `ConnectionIndicator`

Likely GM components:

- `QuartermasterLogin`
- `QuartermasterShell`
- `CampaignStatusCard`
- `PlayerPreview`
- `ProgressionActions`
- `EventLedger`
- `ConfirmationSheet`
- `QuartermasterActionScene`

Requirements:

- Keep snapshot and domain state in clear parents or hooks.
- Keep visual scene refs close to the relevant physical component.
- Do not pass raw animation libraries through dozens of props.
- Use the AnimationDirector and typed scene context.
- Preserve testability.

Split the massive global CSS into logical modules or component styles where supported by the current setup. Keep design tokens centralized.



12. Performance and loading strategy

Using five animation technologies is allowed only with disciplined loading.

12.1 Preloading

- Preload only critical first-arrival assets.
- Load Journal and PageFlip before the user opens the Journal when idle time permits.
- Load later-section assets on navigation intent, idle callbacks, or first section visit.
- Preload the finale only when it becomes relevant.
- Use a safe `requestIdleCallback` fallback.
- Cache parsed Rive files where appropriate.
- Do not make the home route download every artifact and finale asset.

12.2 Visibility

- Pause Lottie loops when hidden or offscreen.
- Stop or settle Rive instances when hidden where practical.
- Pause CSS ambient animation under document-hidden state.
- Pause GSAP timelines on hidden documents unless a server synchronization requires a safe state hold.
- Resume without replaying completed authoritative scenes.

12.3 GPU and memory

- Avoid excessive large transparent WebGL canvases.
- Avoid multiple full-screen canvas layers.
- Release Rive WebGL resources.
- Destroy Lottie instances.
- Destroy PageFlip instances.
- Kill or revert GSAP timelines.
- Remove event listeners.
- Avoid retaining old SplitText DOM.
- Avoid unbounded particle loops.

12.4 Quality tiers

Connect quality to existing full, gentle, and reduced modes.

Full:

- High-detail particles and lighting
- Complete loops

Gentle:

- Lower particle density
- Fewer loops
- Shorter movement

Reduced:

- No ambient animation loops
- Static or brief transitions

Do not use user-agent sniffing. Use capability detection and user preference.

12.5 Performance targets

These are targets, not flaky CI assertions:

- Smooth 60 FPS target on the intended modern desktop hardware.
- Do not fall below a sustained 30 FPS on supported phone viewports during the showcase.
- Avoid repeated long main-thread tasks above 50 ms during ambient loops.
- Keep layout shift minimal after fonts and animation assets load.

- Avoid loading all animation libraries in the first server-rendered bundle.
- Document profiler results and any remaining bottlenecks.



13. Accessibility

Animations must enhance the story without preventing access.

Requirements:

- Respect **prefers-reduced-motion**.
- Preserve the in-app three-mode control.
- Every animation has a reduced equivalent.
- Provide Skip for long ceremonies.
- Provide Pause for the animation trailer.
- No essential information exists only in visual animation.
- Canvas objects have adjacent semantic labels and descriptions.
- StPageFlip pages remain keyboard navigable.
- Focus is never lost behind page turns or overlays.
- Modal focus is trapped appropriately and restored on close.
- Screen readers receive one coherent text copy.
- SplitText does not force character-by-character screen-reader output.
- Avoid rapid flashing.
- Avoid excessive parallax.
- Text remains readable during animation.
- Pointer-only interactions have keyboard alternatives.
- Gesture actions have buttons.
- Color is not the only state indicator.
- The map retains its text alternative.

Run automated and manual accessibility testing.



14. Testing plan

Do not call this complete without tests.

14.1 Unit tests

Test:

- Scene registry
- Scene-name typing
- Mode selection
- Full/gentle/reduced timing policy
- Server synchronization state machine
- Skip behavior before and after server success
- Failure branch selection
- Cancellation
- Queue behavior
- Asset registry
- Fallback selection
- Ownership conflict detection
- Visibility pause policy
- Page model generation from sanitized snapshot
- No secret fields enter page or scene contexts

14.2 GSAP tests

Test:

- Scene builders return timelines or valid controllers.
- Required labels exist.
- Completion resolves once.
- Cancel kills and reverts.
- SplitText reverts.
- A failed operation cannot progress past the safe label.
- Skip while pending stops at the synchronization point.
- Success continues to final state.
- Failure plays reversal or failure scene.

Use deterministic timing or mocked animation clock where practical. Do not make unit tests wait through real ten-second scenes.

14.3 Motion component tests

Test:

- Views have exit states.
- Focus moves to new section.
- Objective dock toggles.
- Modals restore focus.
- Reduced mode chooses reduced variants.
- Shared layout identifiers are stable.
- Mobile navigation remains operable.

14.4 StPageFlip tests

Test:

- Dynamic import is client-only.
- Instance initializes once.
- HTML pages load.
- Updates call `updateFromHtml` rather than duplicating the instance.
- `destroy()` runs on unmount.
- Keyboard next and previous work.
- Chapter selection turns to the correct page.
- Portrait and landscape events update UI.
- Reduced mode uses fallback.
- Page count and current page remain correct after data refresh.

Mock the library in unit tests and cover real behavior in Playwright.

14.5 Rive tests

Test:

- Wrapper renders fallback while loading.
- Local file path is used.
- Missing file produces accessible fallback.
- State-machine input contract is validated.
- Input changes are sent only after instance load.
- Cleanup runs on unmount.
- WebGL failure does not crash the route.
- Reduced mode disables idle behavior.

If only a development sample is available, clearly test the sample separately from production fallbacks.

14.6 Lottie tests

Test:

- `loadAnimation` receives correct renderer, loop, and autoplay settings.
- Data-ready and failure events update status.
- Hidden document pauses.

- Visible document resumes when appropriate.
- Reduced mode does not autoplay loops.
- `destroy()` runs on unmount.
- Segment playback and speed controls work in showcase.

14.7 End-to-end tests

Use Playwright for:

- Landing first arrival
- Replay arrival
- Player access failure
- Player access success
- Journal opening
- Page turn
- Chapter release event
- Map reveal
- Artifact award
- Side-quest discovery
- Ship's Log update
- Finale tease
- Quartermaster login failure
- Quartermaster login success
- Quartermaster actions
- Pause, resume, undo
- Animation showcase
- Trailer replay
- Full, gentle, reduced modes
- Skip while request pending
- Offline and reconnect behavior
- Back and forward navigation
- Fullscreen behavior where supported

Do not mutate production data. Use fresh seeded test databases and existing test utilities.

14.8 Viewports

Validate at minimum:

- 2560 x 1440
- 1920 x 1080
- 1440 x 900
- 430 x 932
- 390 x 844
- 844 x 390

Cover Chromium and WebKit where the existing suite supports them.

14.9 Visual regression

Capture stable screenshots at named scene checkpoints rather than arbitrary milliseconds.

Examples:

- arrival settled
- invitation idle
- access rejected
- access success before journal
- journal open
- page mid-turn if stable enough
- chapter heading complete
- map route complete
- artifact in altar
- quest completed

- finale teased
- Quartermaster dashboard
- showcase trailer title card
- reduced-motion equivalents

Mask or freeze nondeterministic particles for screenshot testing.

14.10 Accessibility tests

Run:

- Existing axe Playwright checks
- Keyboard-only navigation
- Focus order
- Reduced motion
- High text scale
- Screen-reader semantic inspection for split text and page content
- Color contrast
- No hidden duplicate narration

14.11 Cleanup and leak tests

Repeatedly mount and unmount:

- Journal
- Rive object
- Lottie effect
- Animation showcase
- Artifact inspection
- Quartermaster overlay

Assert or inspect:

- No duplicate canvases
- No duplicate event listeners
- No orphan PageFlip wrappers
- No Lottie instances after destroy
- No Rive render loops after unmount
- No active GSAP timelines after route exit
- No console warnings
- No growing memory trend in repeated showcase cycles

14.12 Network and security tests

- No remote animation assets load unexpectedly.
- No hidden chapter content appears in network responses.
- No development route is available in production build.
- No production secret appears in showcase bundles.
- Skip does not bypass authentication or mutation confirmation.
- Animation errors do not expose stack traces to players.

14.13 Build and validation

Run the repository's full validation script, including:

- format check
- lint
- typecheck
- unit tests
- component tests
- end-to-end tests
- accessibility tests
- production build
- restart or smoke validation

Add any new validation steps to the existing `npm run validate` flow where appropriate.



15. Documentation

Create or update:

- `docs/animation/architecture.md`
- `docs/animation/library-ownership.md`
- `docs/animation/scenes.md`
- `docs/animation/assets.md`
- `docs/animation/rive-assets.md`
- `docs/animation/lottie-assets.md`
- `docs/animation/testing.md`
- `docs/animation/performance.md`
- `docs/animation/showcase.md`
- existing design system documentation
- existing asset-license documentation
- known limitations
- local development instructions
- Codex handoff

Document:

- Why each library is used
- Which elements each library owns
- How to add a new scene
- How to add a Rive asset
- How to add a Lottie asset
- How to add a Journal page type
- How to use the showcase
- How to test all modes
- How to profile performance
- What remains dependent on designer-created `.riv` or Bodymovin assets

Do not claim unavailable art assets are complete.



16. Required deliverables

The completed work must include:

1. Dependency migration from `framer-motion` to `motion`.
2. GSAP and React integration.
3. Central cinematic director.
4. Removal or retirement of timer-only visual orchestration.
5. StPageFlip Journal implementation.
6. Appropriate Side-Quest Ledger page-flip implementation.
7. Motion-based interface transitions across all sections.
8. Rive wrappers and typed asset contracts.
9. At least one valid demonstrated Rive state-machine integration in development, with honest production fallback status if original assets are unavailable.
10. Lottie wrapper and multiple original or licensed local effects.
11. Rebuilt first arrival.
12. Rebuilt player access scene.
13. Rebuilt chapter-release flagship ceremony.
14. Rebuilt map reveal.
15. Rebuilt artifact ceremony.

16. Rebuilt quest animations.
17. Rebuilt Ship's Log entry animation.
18. Rebuilt finale tease and mechanism shell.
19. Rebuilt Quartermaster login.
20. Distinct Quartermaster action animations.
21. Development-only **TEST ANIMATIONS** button.
22. `/dev/animations` showcase.
23. Full cinematic trailer.
24. All three motion modes.
25. Cleanup, fallback, and error behavior.
26. Complete automated tests.
27. Visual validation at required viewports.
28. Documentation.
29. Version update.
30. Final commit and validation receipt according to repository workflow.



17. Quality bar

Reject your own implementation if any of these are true:

- The libraries are installed but barely used.
- Most scenes remain timer-driven class toggles.
- Long animations contain dead air.
- Page flipping is still a slight **rotateY** animation.
- Live writing is still only whole-block opacity.
- The Quartermaster still shows the same generic overlay for every action.
- Rive is claimed without a valid `.riv` runtime demonstration.
- Lottie files fail to load or are remote placeholders.
- Production routes download the full showcase.
- Animations reveal state too early.
- Skip bypasses a server gate.
- Reduced motion simply disables everything without preserving narrative meaning.
- Mobile becomes unusable.
- Canvas content has no semantic alternative.
- Cleanup leaks instances.
- Tests use arbitrary sleeps instead of scene checkpoints where avoidable.
- There are hydration errors, console errors, or uncaught promise rejections.
- Copyrighted Sea of Thieves material is added.
- Private surprise content is committed while the repository is public.

The finished site should feel physically authored, narratively coherent, and substantially more detailed than the current animation pass.



18. Final validation receipt

At completion, provide a detailed final response containing:

- Branch
- Starting commit
- Ending commit
- Version before and after
- Dependency changes with exact versions
- Major architecture changes
- Page-by-page implementation summary
- Animation showcase route and access instructions

- Trailer description
- Rive asset status and any honest limitations
- Lottie asset list
- Test counts and results
- Playwright viewport and browser coverage
- Accessibility results
- Production-build result
- Performance observations
- Cleanup verification
- Security and privacy verification
- Asset-license updates
- Exact changed files grouped by purpose
- Remaining limitations
- Confirmation that remote branch matches local HEAD if pushed

Do not provide a triumphant vague summary. Provide evidence.

END CODEX PROMPT